TaxonVision-MLOps

Comprehensive Technical Implementation Guide

Systems Architecture, Infrastructure, and Codebase Reference

Benjamin Förster

October 11, 2026

Abstract

This technical report serves as an exhaustive, implementation-level architectural manual and code base reference for the TaxonVision-MLOps platform. Designed to document every structural component, configuration file, deployment manifest, mathematical loss formulation, and runtime daemon in the active repository, this document provides complete visibility into the physical server topology (`hive-mind` running Ubuntu 26.04.1 LTS, NVIDIA Driver 615.71.09, CUDA 13.4, and the NVIDIA GeForce RTX 5070 Ti with Blackwell SM 120), the lightweight Kubernetes (k3s) cluster partitioned into `jenkins-ci` and `taxon-vision` namespaces, the Cloudflare Zero Trust ingress tunnel, and the four-slot NVIDIA GPU time-slicing device plugin. Furthermore, this manual dissects the pure Pixi multi-environment toolchain, the automated DagsHub storage remote for Data Version Control (DVC) and MLflow tracking, the Cui et al. Class-Balanced Loss dynamics on biological long-tail Pareto distributions, the Optuna Bayesian hyperparameter optimisation engine, Split Conformal Prediction, Energy-Based Out-of-Distribution rejection, and the unified FastAPI and HTMX operator dashboard. Every configuration parameter, file extension, and container specification is strictly analysed based upon active codebase evidence.

Contents

Abstract **i**

1 Repository Architecture, Directory Hierarchy, and File Extension Semantics **1**
1.1 Root Directory Topology 1
1.2 Configuration Manifests and Semantics 1
1.3 Specialised File Formats and Storage Conventions 1

2 Bare-Metal Server Environment and Pixi Toolchain Interlock **2**
2.1 Physical Compute Node Baseline 2
2.2 Multi-Target Toolchain and Pixi Environment Isolation 2
2.3 Command Orchestration via Justfile 2

3 Lightweight Kubernetes (k3s) Cluster Topology and Security RBAC **3**
3.1 Cluster Architecture and Network Edge 3
3.2 Namespace Partitioning and Multi-Tenant Segregation 3
3.3 Role-Based Access Control (RBAC) Specification 3

4 GPU Virtualisation, NVIDIA Device Plugin, and VRAM Quotas **4**
4.1 The Single-GPU Multi-Tenancy Challenge 4
4.2 NVIDIA Device Plugin Time-Slicing Configuration 4
4.3 Deterministic VRAM Budget Capping and OOM Prevention 4

5 Containerisation Architecture: Dockerfile and Composition Manifests **5**
5.1 Multi-Stage Container Construction Strategy 5
5.2 Auxiliary Services and Docker Compose Topology 5
5.3 Container Hardening and Privilege Minimisation 5

6 Kubernetes Workloads, Service Manifests, and Rolling Updates **6**
6.1 API Deployment Specification 6
6.2 Health Probes and Zero-Downtime Rolling Restarts 6
6.3 Kubernetes Service Abstraction and Routing 6

7 Declarative Jenkins Pipeline and Ephemeral Kubernetes Agents **7**
7.1 Declarative Pipeline Architecture and Pod Template 7
7.2 Ephemeral Agent Lifecycle and Cache Volume Mounting 7
7.3 Stage-by-Stage Execution and Promotion Gates 7

8 Data Ingestion Engine, Zero-Copy Arrow Streaming, and License Audits **8**
8.1 iNaturalist API Ingestion and Query Mechanics 8
8.2 Zero-Copy Apache Arrow Streaming Pipeline 8
8.3 Open License Verification and Attribution Retention 8

9 Data Version Control (DVC) and Remote Storage Topology **9**
9.1 DVC Pipeline Graph and Cryptographic Manifests 9

9.2 DagsHub S3-Compatible Remote Storage Architecture 9
9.3 Synchronization Lifecycles in CI/CD and Local Workstations 9

10 Domain Modelling, DarwinCore Standards, and Taxonomic Trees 10
10.1 DarwinCore Taxon Schema and Catalog Implementation 10
10.2 Observation Entities and Spatiotemporal Metadata 10
10.3 License Verification Domain Invariants 10

11 Vision Backbone Factory and Representation Extraction Caching 11
11.1 Foundation Vision Backbone Portfolio 11
11.2 Extensible Feature Extractor Factory Architecture 11
11.3 Representation Extraction and Disk-Caching Strategy 11

12 Class-Balanced Loss Formulation and Long-Tail Gradient Dynamics 12
12.1 The Biological Long-Tail Pareto Distribution 12
12.2 Effective Number of Samples Mathematical Derivation 12
12.3 PyTorch Implementation and Numerical Stabilisation 12

13 Model Head Training and Convergence Callbacks 13
13.1 Introduction to Decoupled Linear Probing 13
13.2 TaxonClassifier Architecture 13
13.2.1 Frozen Vision Backbones 13
13.2.2 Linear Projection and Regularisation 14
13.2.3 Dual Execution Pathways 14
13.3 Training and Validation Loop Mechanics 14
13.3.1 Gradient Optimisation and PyTorch Steps 14
13.3.2 Metric Computation for Imbalanced Distributions 15
13.4 Convergence Callbacks and Early Stopping 15
13.4.1 Dynamic Early Stopping 15
13.4.2 Learning Rate Schedulers 15
13.4.3 Memory Management and Hardware Optimisation 16
13.5 Checkpoint Management and Atomic Serialisation 16
13.5.1 Checkpoint Anatomy 16
13.5.2 Atomic File Saving Strategy 16
13.6 Summary 17

14 Bayesian Hyperparameter Optimization with Optuna TPE and ASHA 18
14.1 Bayesian Optimization Principles and TPE Mechanics 18
14.2 Search Space Parameterisation and Objective Formulation 18
14.3 Asynchronous Successive Halving (ASHA) Pruning in Practice 18

15 Experiment Tracking, Cryptographic Lineage, and MLflow Model Registry 19
15.1 Experiment Tracking Architecture on DagsHub 19
15.2 Deterministic Lifecycle Management via RAII Scopes 19
15.3 Cryptographic Lineage Capture and Automated Model Promotion 19

16 Epistemic Safety: Split Conformal Prediction and Energy-Based OOD Rejection 20
16.1 The Epistemic Safety Problem in Biological Vision 20
16.2 Split Conformal Prediction and Statistical Coverage Bounds 20
16.3 Helmholtz Free Energy Out-of-Distribution Detection 20

17 Active Learning Strategy and Priority Triage Queue 21
17.1 Uncertainty-Based Query Acquisition Mechanics 21
17.2 Priority Triage Queue Implementation 21
17.3 Closing the Continuous Retraining Annotation Loop 21

18 ONNX Export, Zero-Allocation Runtime, and INT8 Quantization 22

18.1 Computational Graph Export and Dynamic Axes 22
18.2 Zero-Allocation C++ Inference Execution Engine 22
18.3 INT8 Quantization Calibration and Class Activation Mapping 22

19 FastAPI REST Service and Server-Rendered HTMX Operator Dashboard 23
19.1 Unified REST and Web Application Architecture 23
19.2 API Schemas, Endpoints, and Inference Routing 23
19.3 Server-Rendered HTMX Operator Interface 23

20 Real-Time Telemetry, Prometheus Instrumentation, and Drift Detection 24
20.1 Prometheus Telemetry Instrumentation and Metrics Design 24
20.2 Serving Latency and Conformal Set Monitoring 24
20.3 Embedding Representation Drift via Wasserstein and MMD 24

21 Quality Assurance: Pytest Matrix, Invariant Testing, and Complexity Budgets 25
21.1 Testing Architecture and Test Matrix Topology 25
21.2 Mathematical Invariant Assertions and Property-Based Testing 25
21.3 Cognitive Complexity Budgets and Pre-Commit Quality Gates 25

22 Open Knowledge Format (OKF v0.2) and Master Agent Routing 26
22.1 Open Knowledge Format (OKF v0.2) Schema and Trust Tiers 26
22.2 Multi-Agent Governance and Decision Routing Matrices 26
22.3 Automated Validation Scripts and D3 Knowledge Graph Visualization 26

23 Developer Operations, Environment Setup, and Engineering Runbooks 27
23.1 Developer Workstation Onboarding and Environment Setup 27
23.2 Dataset Synchronization and Remote Storage Runbook 27
23.3 Cluster Operations, GPU Debugging, and Emergency Rollback 27

Chapter 1

Repository Architecture, Directory Hierarchy, and File Extension Semantics

1.1 Root Directory Topology

1.2 Configuration Manifests and Semantics

1.3 Specialised File Formats and Storage Conventions

Chapter 2

Bare-Metal Server Environment and Pixi Toolchain Interlock

2.1 Physical Compute Node Baseline

2.2 Multi-Target Toolchain and Pixi Environment Isolation

2.3 Command Orchestration via Justfile

Chapter 3

Lightweight Kubernetes (k3s) Cluster Topology and Security RBAC

3.1 Cluster Architecture and Network Edge

3.2 Namespace Partitioning and Multi-Tenant Segregation

3.3 Role-Based Access Control (RBAC) Specification

Chapter 4

GPU Virtualisation, NVIDIA Device Plugin, and VRAM Quotas

4.1 The Single-GPU Multi-Tenancy Challenge

4.2 NVIDIA Device Plugin Time-Slicing Configuration

4.3 Deterministic VRAM Budget Capping and OOM Prevention

Chapter 5

Containerisation Architecture: Dockerfile and Composition Manifests

5.1 Multi-Stage Container Construction Strategy

5.2 Auxiliary Services and Docker Compose Topology

5.3 Container Hardening and Privilege Minimisation

Chapter 6

Kubernetes Workloads, Service Manifests, and Rolling Updates

6.1 API Deployment Specification

6.2 Health Probes and Zero-Downtime Rolling Restarts

6.3 Kubernetes Service Abstraction and Routing

Chapter 7

Declarative Jenkins Pipeline and Ephemeral Kubernetes Agents

7.1 Declarative Pipeline Architecture and Pod Template

7.2 Ephemeral Agent Lifecycle and Cache Volume Mounting

7.3 Stage-by-Stage Execution and Promotion Gates

Chapter 8

Data Ingestion Engine, Zero-Copy Arrow Streaming, and License Audits

8.1 iNaturalist API Ingestion and Query Mechanics

8.2 Zero-Copy Apache Arrow Streaming Pipeline

8.3 Open License Verification and Attribution Retention

Chapter 9

Data Version Control (DVC) and Remote Storage Topology

9.1 DVC Pipeline Graph and Cryptographic Manifests

9.2 DagsHub S3-Compatible Remote Storage Architecture

9.3 Synchronization Lifecycles in CI/CD and Local Workstations

Chapter 10

Domain Modelling, DarwinCore Standards, and Taxonomic Trees

10.1 DarwinCore Taxon Schema and Catalog Implementation

10.2 Observation Entities and Spatiotemporal Metadata

10.3 License Verification Domain Invariants

Chapter 11

Vision Backbone Factory and Representation Extraction Caching

11.1 Foundation Vision Backbone Portfolio

11.2 Extensible Feature Extractor Factory Architecture

11.3 Representation Extraction and Disk-Caching Strategy

Chapter 12

Class-Balanced Loss Formulation and Long-Tail Gradient Dynamics

12.1 The Biological Long-Tail Pareto Distribution

12.2 Effective Number of Samples Mathematical Derivation

12.3 PyTorch Implementation and Numerical Stabilisation

Chapter 13

Model Head Training and Convergence Callbacks

13.1 Introduction to Decoupled Linear Probing

The computational cost of fine-tuning large-scale vision foundation models across thousands of granular biological classes renders traditional end-to-end backpropagation intractable for rapid experimentation. TaxonVision-MLOps addresses this limitation by adopting a decoupled linear probing philosophy. This approach strictly separates the extraction of high-dimensional visual priors, which are computed via pre-trained, frozen vision transformer or convolutional neural network (CNN) backbones, from the taxonomic categorisation task. By restricting trainable parameters strictly to a terminal, lightweight linear projection head, the platform accelerates optimisation cycles from hours to mere seconds. This architecture fundamentally eliminates the risk of catastrophic forgetting of generalized visual representations whilst simultaneously mitigating gradient instability and curbing excessive GPU memory consumption.

13.2 TaxonClassifier Architecture

The core of this decoupled methodology is crystallised within the `TaxonClassifier` module, located at `src/taxon_vision/models/head.py`. This class encapsulates the dual-stage evaluation pathway and enforces rigorous regularisation.

13.2.1 Frozen Vision Backbones

Upon instantiation, the `TaxonClassifier` accepts a pre-trained feature extractor backbone. To guarantee parameter immutability and prevent the propagation of noisy gradients from heavily imbalanced target domains into the robustly pre-trained feature space, the model traverses all backbone parameters and forcefully disables gradient tracking:

```
for param in self.backbone.parameters():
    param.requires_grad = False
```

This explicitly shields the backbone. The computational complexity of the forward pass remains constant with respect to the number of training iterations, represented as $\mathcal{O}(T \times B \times C_{\text{feat}})$, where T denotes the total number of training steps, B is the batch size, and C_{feat} signifies the fixed computational cost of a backbone forward pass.

13.2.2 Linear Projection and Regularisation

Categorisation is performed by a sequentially structured trainable projection head. To combat over-fitting and prevent the co-adaptation of neurons on high-dimensional embeddings, a stochastic regularisation layer is injected prior to the linear projection mapping. Specifically, a Dropout layer with a default probability $p = 0.2$ is employed:

```
self.head = nn.Sequential(
    nn.Dropout(p=dropout),
    nn.Linear(feature_dim, num_classes),
)
```

This forces the linear weights to rely upon a diverse constellation of visual features rather than overly depending upon any single idiosyncratic activation vector.

13.2.3 Dual Execution Pathways

The architecture facilitates both end-to-end inference and accelerated training via dual forward methodologies:

1. **End-to-End Execution (`forward`):** This standard pathway ingests 4D raw image tensors of shape (B, C, H, W). It delegates feature extraction to the `forward_features` method, which securely wraps the backbone computation within a `torch.no_grad()` context manager to ensure spatial and temporal memory efficiency. The resulting embeddings are subsequently channelled through the head to produce unnormalised class logits.
2. **Rapid Optimisation Pathway (`forward_head`):** During hyperparameter tuning or iterative head training, computing the backbone outputs repeatedly is entirely redundant. The `forward_head` method directly ingests pre-computed 2D embedding tensors of shape (B, D) and projects them to class logits. This circumvention enables near-instantaneous stochastic gradient descent updates.

13.3 Training and Validation Loop Mechanics

The operational execution of the training epochs is orchestrated by the subroutines defined within `src/taxon_vision/models/training/loops.py`. These functions codify the forward, backward, and optimisation steps alongside rigorous metric computation tailored for heavily imbalanced taxonomic distributions.

13.3.1 Gradient Optimisation and PyTorch Steps

The `train_head_epoch` function governs a single iteration over the dataset. To ensure state consistency, it explicitly places the encompassing model in `train()` mode, whilst overriding the backbone state to `eval()`, thereby locking internal statistics such as batch normalisation running means and variances.

Within the iterative loop processing minibatches of inputs and labels, the optimisation sequence adheres to standard PyTorch semantics:

1. `optimizer.zero_grad()`: Flushes the accumulated gradient buffers to prevent residual contamination from antecedent steps.
2. `outputs = model(inputs)`: Executes the forward pass to produce unnormalised logits.
3. `loss = criterion(outputs, labels)`: Computes the differentiable loss landscape via objective functions such as the Class-Balanced Loss.
4. `loss.backward()`: Computes the gradient of the loss with respect to the trainable head parameters using the chain rule. Notably, owing to the `requires_grad = False` attribute,

this backpropagation halts precisely at the boundary of the linear head, achieving significant computational savings.

5. `optimizer.step()`: Updates the parameter weights via algorithms such as AdamW or Stochastic Gradient Descent (SGD).

To ensure numerical stability during gradient accumulation and mixed-precision operations, developers may leverage `torch.cuda.amp.autocast` wrappers around the forward pass and loss computation, protecting against underflow or overflow in half-precision (FP16) arithmetic.

13.3.2 Metric Computation for Imbalanced Distributions

Taxonomic datasets universally exhibit extreme long-tail Pareto distributions, rendering raw accuracy a highly misleading indicator of generalisation performance. Consequently, the `evaluate_head` function calculates multiple axes of performance:

- **Top-1 Categorical Accuracy:** The fundamental ratio of correctly classified samples to the total sample volume. Whilst mathematically robust, it is easily dominated by the majority classes.
- **Balanced Accuracy:** An unweighted arithmetic mean of recall statistics calculated independently for each taxonomic class. This enforces an equal penalty for misclassifying a rare species as a common species, thus yielding a more realistic proxy for true generalisation.
- **Macro-Averaged F_1 Score:** The harmonic mean of precision and recall computed uniformly across all classes. It strongly penalises models that neglect the tail of the distribution, serving as the primary metric for convergence monitoring and early stopping criteria.

13.4 Convergence Callbacks and Early Stopping

The continuous orchestration of training epochs is governed by `src/taxon_vision/models/training/runner.py`, which leverages sophisticated heuristics to regulate learning rates, monitor validation plateaus, and perform vital memory management. These configurations are formalised via data classes within `src/taxon_vision/models/training/types.py`.

13.4.1 Dynamic Early Stopping

Extended training on finite cached embeddings carries an inherent risk of over-fitting to the training manifold at the expense of generalisation. To combat this, the `HeadTrainingConfig` structure implements dynamic early stopping based upon validation plateau detection.

The mechanism monitors the validation macro F_1 score (or validation loss) iteratively. An `early_stopping_patience` integer dictates the maximum number of consecutive epochs permissible without observing an improvement in the monitored metric. If this threshold is breached, training is immediately terminated. This ensures the retention of the most generalisable weight configuration and significantly reduces total compute expenditure.

13.4.2 Learning Rate Schedulers

To traverse the loss landscape efficiently, the learning rate must adapt dynamically. Two primary scheduler philosophies are integrated:

- **ReduceLROnPlateau:** Configured via the `reduce_lr_patience` attribute. Upon detecting a stagnation in validation loss beyond the defined epoch threshold, this scheduler multiplicatively attenuates the learning rate, enabling the optimiser to descend into narrower, more refined local minima that were previously skipped due to high-velocity updates.
- **Cosine Annealing with Linear Warmup:** This strategy linearly increases the learning rate during initial iterations to avoid rapid divergence caused by massive gradient updates in

unstable regions. Subsequently, it decays the rate following a cosine curve, allowing smooth convergence toward the end of the predefined epoch limit.

13.4.3 Memory Management and Hardware Optimisation

Given the constraints of GPU VRAM (e.g., the 8 GB limit of the NVIDIA RTX 5070 Ti) and the aggressive time-slicing required for concurrent workloads, deterministic memory management is strictly enforced. The runner script explicitly invokes Python garbage collection `gc.collect()` and explicitly clears the CUDA cache via `torch.cuda.empty_cache()` between epoch transitions. This mitigates latent memory fragmentation and ensures maximal VRAM availability for successive matrix multiplications.

13.5 Checkpoint Management and Atomic Serialisation

The persistence layer, detailed within `src/taxon_vision/models/training/checkpoint.py`, guarantees the seamless translation of trained weights into highly available inference artifacts without risking deployment corruption.

13.5.1 Checkpoint Anatomy

A comprehensive checkpoint dictionary encapsulates far more than raw parameter weights. It is explicitly serialised to encompass:

- `model_state_dict`: The precise floating-point weights and biases of the linear projection head.
- `optimizer_state_dict`: The internal momentum and variance buffers of the AdamW optimiser, ensuring training can be resumed seamlessly without experiencing massive gradient spikes.
- **Epoch and Metric Metadata:** The specific epoch number and corresponding peak validation metrics (e.g., maximum macro F_1) achieved.
- **Taxonomic Label Mappings:** An explicit dictionary mapping raw integer class indices to human-readable Darwin Core taxonomic nomenclature.
- **Backbone Metadata:** An explicit string identifier (e.g., `dinov3`, `bioclip-2`) denoting the exact feature extractor required to produce valid embeddings for the saved linear head.

13.5.2 Atomic File Saving Strategy

In cloud-native, Kubernetes-orchestrated environments (`k3s`), containers face a constant threat of arbitrary preemption and rescheduling. A naive file write operation (`torch.save`) to the primary checkpoint file is inherently vulnerable; if the process is terminated mid-write, the resultant file will be irrecoverably corrupted, subsequently crashing the dependent inference daemon during cold starts.

To guarantee transactional integrity, the `_save_checkpoint` method employs an atomic rename strategy. The serialisation process first writes the entire state dictionary to a temporary, hidden file upon the disk (e.g., `.tmp`). Only upon the successful, complete materialisation of this file does the operating system execute an atomic rename operation (`os.replace`) to overwrite the active checkpoint file. This guarantees that any consuming process only ever reads a fully intact, complete checkpoint.

Finally, upon successfully persisting the updated model weights, the function invokes `get_classifier.cache_clear()`. This forcibly invalidates the `lru_cache` of the active FastAPI inference service, ensuring that all subsequent classification requests immediately load and evaluate the newly optimised linear head without necessitating a manual pod restart.

Table 13.1: Key configuration parameters governing linear head training.

Parameter	Default	Description
`epochs`	10	Absolute maximum number of training iterations.
`batch_size`	64	Number of samples processed concurrently prior to a gradient update step.
`early_stopping_patience`	4	Number of stagnant epochs permitted before termination.
`reduce_lr_patience`	2	Epoch threshold before learning rate decay is triggered.
`noise_std`	0.0	Standard deviation of Gaussian noise injected into embeddings for manifold regularisation.

13.6 Summary

The structural design surrounding the `TaxonClassifier` enables agile, memory-efficient adaptation of state-of-the-art vision foundations into highly specialised biological taxonomic classifiers. The rigorous atomic serialisation protocol, coupled with aggressive early stopping upon metrics specifically tuned for long-tail distributions, forms a highly resilient mechanism capable of operating cleanly within distributed, preemption-heavy Kubernetes clusters.

Chapter 14

Bayesian Hyperparameter Optimization with Optuna TPE and ASHA

14.1 Bayesian Optimization Principles and TPE Mechanics

14.2 Search Space Parameterisation and Objective Formulation

14.3 Asynchronous Successive Halving (ASHA) Pruning in Practice

Chapter 15

Experiment Tracking, Cryptographic Lineage, and MLflow Model Registry

15.1 Experiment Tracking Architecture on DagsHub

15.2 Deterministic Lifecycle Management via RAII Scopes

15.3 Cryptographic Lineage Capture and Automated Model Promotion

Chapter 16

Epistemic Safety: Split Conformal Prediction and Energy-Based OOD Rejection

16.1 The Epistemic Safety Problem in Biological Vision

16.2 Split Conformal Prediction and Statistical Coverage Bounds

16.3 Helmholtz Free Energy Out-of-Distribution Detection

Chapter 17

Active Learning Strategy and Priority Triage Queue

17.1 Uncertainty-Based Query Acquisition Mechanics

17.2 Priority Triage Queue Implementation

17.3 Closing the Continuous Retraining Annotation Loop

Chapter 18

ONNX Export, Zero-Allocation Runtime, and INT8 Quantization

18.1 Computational Graph Export and Dynamic Axes

18.2 Zero-Allocation C++ Inference Execution Engine

18.3 INT8 Quantization Calibration and Class Activation Mapping

Chapter 19

FastAPI REST Service and Server-Rendered HTMX Operator Dashboard

19.1 Unified REST and Web Application Architecture

19.2 API Schemas, Endpoints, and Inference Routing

19.3 Server-Rendered HTMX Operator Interface

Chapter 20

Real-Time Telemetry, Prometheus Instrumentation, and Drift Detection

20.1 Prometheus Telemetry Instrumentation and Metrics Design

20.2 Serving Latency and Conformal Set Monitoring

20.3 Embedding Representation Drift via Wasserstein and MMD

Chapter 21

Quality Assurance: Pytest Matrix, Invariant Testing, and Complexity Budgets

21.1 Testing Architecture and Test Matrix Topology

21.2 Mathematical Invariant Assertions and Property-Based Testing

21.3 Cognitive Complexity Budgets and Pre-Commit Quality Gates

Chapter 22

Open Knowledge Format (OKF v0.2) and Master Agent Routing

22.1 Open Knowledge Format (OKF v0.2) Schema and Trust Tiers

22.2 Multi-Agent Governance and Decision Routing Matrices

22.3 Automated Validation Scripts and D3 Knowledge Graph Visualization

Chapter 23

Developer Operations, Environment Setup, and Engineering Runbooks

23.1 Developer Workstation Onboarding and Environment Setup

23.2 Dataset Synchronization and Remote Storage Runbook

23.3 Cluster Operations, GPU Debugging, and Emergency Rollback

Bibliography

[1] Anastasios N. Angelopoulos and Stephen Bates. A gentle introduction to conformal prediction and distribution-free uncertainty quantification. *arXiv preprint arXiv:2107.07511*, 2021.

[2] Jordan T. Ash, Chicheng Zhang, Akshay Krishnamurthy, John Langford, and Alekh Agarwal. Deep batch active learning by diverse gradient embeddings. In *International Conference on Learning Representations (ICLR)*, 2020.

[3] Yin Cui, Menglin Jia, Tsung-Yi Lin, Yang Song, and Serge Belongie. Class-balanced loss based on effective number of samples. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 9268–9277. IEEE, 2019.

[4] Pengzhen Ren, Yun Xiao, Xiaojun Chang, Po-Yao Huang, Zhihui Li, Brij B. Gupta, Xiaojiang Chen, and Xinwang Wang. A survey of deep active learning. *ACM Computing Surveys (CSUR)*, 54(9):1–40, 2021.

[5] Samuel Stevens, Jiaman Wu, Matthew J. Thompson, Elizabeth G. Campolongo, Chan Hee Song, David E. Carlyn, Li Dong, Wasila M. Dahdul, Charles Stewart, Tanya Berger-Wolf, Wei-Lun Chao, and Yu Su. Bioclip: A vision foundation model for the tree of life. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 19412–19424. IEEE, 2024.

[6] Grant Van Horn, Oisin Mac Aodha, Yang Song, Yin Cui, Chen Sun, Alex Shepard, Hartwig Adam, Pietro Perona, and Serge Belongie. The inaturalist species classification and detection dataset. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 8769–8778. IEEE, 2018.